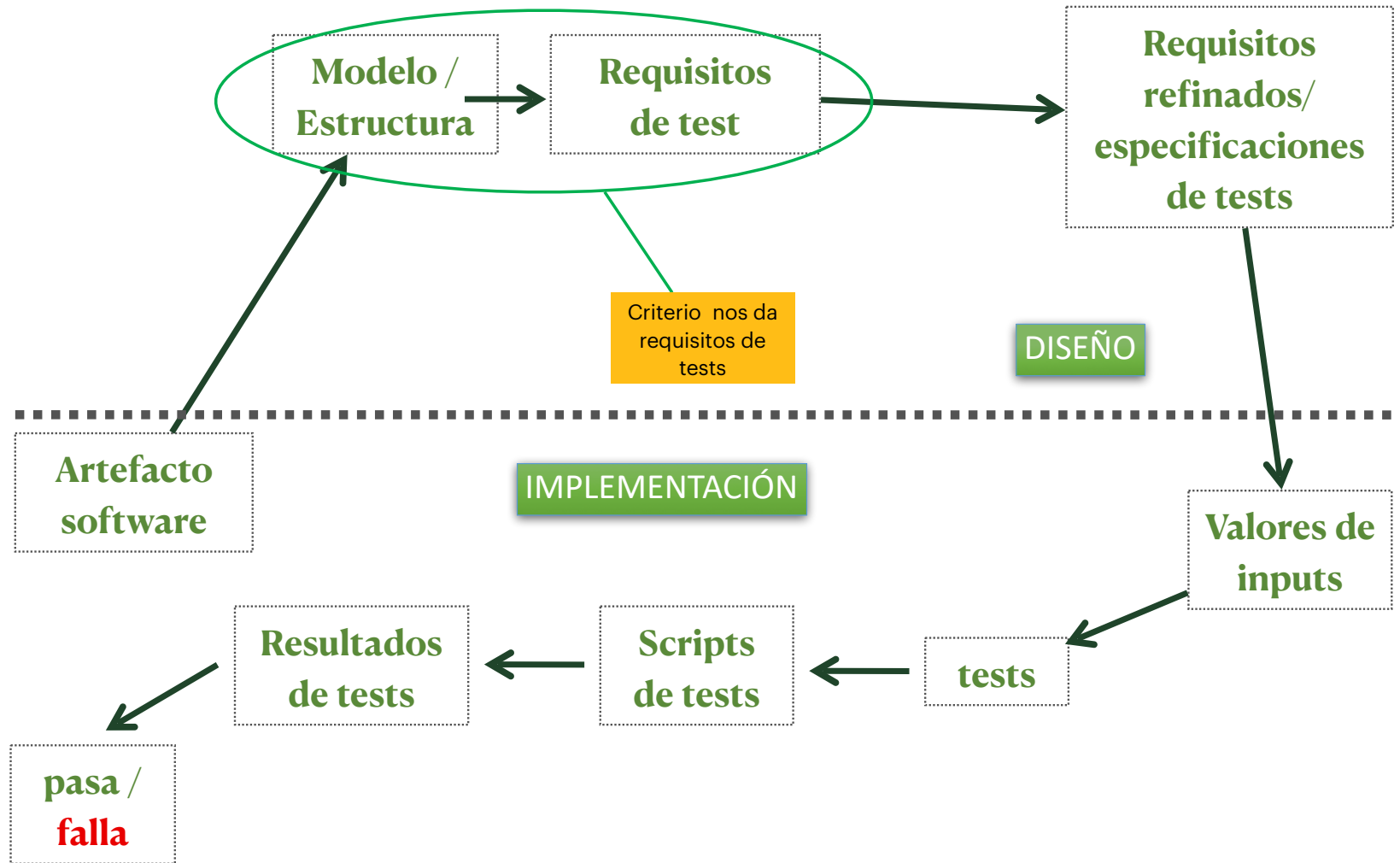


Diseño de tests basado en criterios

Valeria Bengolea

Estas transparencias están basadas en las desarrolladas por Ammann & Offutt como acompañamiento de su libro Introduction to Software Testing (2nd Edition) y por Manuel Núñez (Versión en castellano)

Model-Driven Test Diseño



Criterios de cobertura

La tarea del testear es simple: definir un modelo del software y encontrar formas de recorrerlo

Requisito de test: un elemento específico de un artefacto software que un test debe satisfacer o cubrir

Criterios de cobertura

La tarea del testear es simple: definir un modelo del software y encontrar formas de recorrerlo

Requisito de test: un elemento específico de un artefacto software que un test debe satisfacer o cubrir

Criterios de cobertura. Colección de reglas que imponen los requisitos de test sobre el conjunto de tests.

Criterios de cobertura

La tarea del testear es simple: definir un modelo del software y encontrar formas de recorrerlo

Requisito de test: un elemento específico de un artefacto software que un test debe satisfacer o cubrir

Criterios de cobertura. Colección de reglas que imponen los requisitos de test sobre el conjunto de tests.

Los investigadores han definido muchos criterios pero se pueden reducir a muchos menos si los agrupamos por la estructura subyacente.

Crterios basados en estructuras

Estructuras: Consideramos cuatro formas de modelar el software.

Criterios basados en estructuras

Estructuras: Consideramos cuatro formas de modelar el software.

1. Caracterización del dominio de inputs (conjuntos).

A: {0, 1, >1}
B: {600, 700, 800}
C: {ATM, RM, BCN, OTRO}

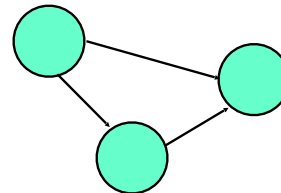
Criterios basados en estructuras

Estructuras: Consideramos cuatro formas de modelar el software.

1. Caracterización del dominio de inputs (conjuntos).

A: {0, 1, >1}
B: {600, 700, 800}
C: {ATM, RM, BCN, OTRO}

2. Grafos.



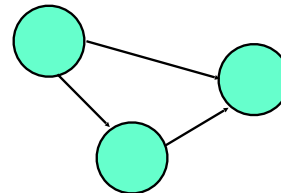
Criterios basados en estructuras

Estructuras: Consideramos cuatro formas de modelar el software.

1. Caracterización del dominio de inputs (conjuntos).

A: {0, 1, >1}
B: {600, 700, 800}
C: {ATM, RM, BCN, OTRO}

2. Grafos.



3. Expresiones lógicas.

(not X or not Y) and A and B

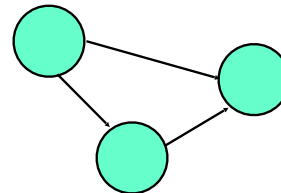
Criterios basados en estructuras

Estructuras: Consideramos cuatro formas de modelar el software.

1. Caracterización del dominio de inputs (conjuntos).

A: {0, 1, >1}
B: {600, 700, 800}
C: {ATM, RM, BCN, OTRO}

2. Grafos.



3. Expresiones lógicas.

(not X or not Y) and A and B

4. Estructuras sintácticas (gramáticas).

```
if (x > y)
  z = x - y;
else
  z = 2 * x;
```

¿De donde salen las estructuras?

Las **estructuras** se pueden **extraer** de muchos artefactos **software**.

¿De donde salen las estructuras?

Las **estructuras** se pueden **extraer** de muchos artefactos **software**.

Los grafos se pueden extraer de casos de uso UML, máquinas de estados finitos, código fuente,...

¿De donde salen las estructuras?

Las **estructuras** se pueden **extraer** de muchos artefactos **software**.

Los grafos se pueden extraer de casos de uso UML, máquinas de estados finitos, código fuente,...

Las expresiones lógicas se pueden extraer de decisiones en el código fuente, de las guardas en transiciones, de condicionales en casos de uso, especificaciones, ...

Visión antigua: cajas coloreadas

Visión antigua: cajas coloreadas

Black Box Testing: Derivar tests desde descripciones externas al código, incluyendo especificaciones, requisitos y diseño

Visión antigua: cajas coloreadas

Black Box Testing: Derivar tests desde descripciones externas al código, incluyendo especificaciones, requisitos y diseño

White Box Testing: Derivar tests de la estructura interna del código fuente, incluyendo condiciones y sentencias

Visión antigua: cajas coloreadas

Black Box Testing: Derivar tests desde descripciones externas al código, incluyendo especificaciones, requisitos y diseño

White Box Testing: Derivar tests de la estructura interna del código fuente, incluyendo condiciones y sentencias

Model Based Testing: Derivar tests de un modelo del software (ej. un diagrama UML)

Visión antigua: cajas coloreadas

Black Box Testing: Derivar tests desde descripciones externas al código, incluyendo especificaciones, requisitos y diseño

White Box Testing: Derivar tests de la estructura interna del código fuente, incluyendo condiciones y sentencias

Model Based Testing: Derivar tests de un modelo del software (ej. un diagrama UML)

Model-Driven Test Diseño le resta importancia a estas distinciones, y pone el foco en los modelos usados para derivar los tests

Visión antigua: cajas coloreadas

Black Box Testing: Derivar tests desde descripciones externas al código, incluyendo especificaciones, requisitos y diseño

White Box Testing: Derivar tests de la estructura interna del código fuente, incluyendo condiciones y sentencias

Model Based Testing: Derivar tests de un modelo del software (ej. un diagrama UML)

Model-Driven Test Diseño le resta importancia a estas distinciones, y pone el foco en los modelos usados para derivar los tests

Ejemplo: Cobertura de Caramelos

Ejemplo: Cobertura de Caramelos



Ejemplo: Cobertura de Caramelos

Tenemos seis sabores:

1. Limón.
2. Pistacho.
3. Melón.
4. Pera
5. Mandarina.
6. Damasco.



Ejemplo: Cobertura de Caramelos

Tenemos **seis sabores**:

1. Limón.
2. Pistacho.
3. Melón.
4. Pera
5. Mandarina.
6. Damasco.

Tenemos **cuatro colores**:

1. **Amarillo** (Limón, damasco).
2. **Verde** (Pistacho).
3. **Naranja** (Melón, mandarina).
4. Blanco (Pera)



Ejemplo: Cobertura de Caramelos

Tenemos **seis sabores**:

1. Limón.
2. Pistacho.
3. Melón.
4. Pera
5. Mandarina.
6. Damasco.

Criterios de cobertura posibles:

Tenemos **cuatro colores**:

1. **Amarillo** (Limón, damasco).
2. **Verde** (Pistacho).
3. **Naranja** (Melón, mandarina).
4. Blanco (Pera)



Ejemplo: Cobertura de Caramelos

Tenemos **seis sabores**:

1. Limón.
2. Pistacho.
3. Melón.
4. Pera
5. Mandarina.
6. Damasco.

Tenemos **cuatro colores**:

1. **Amarillo** (Limón, damasco).
2. **Verde** (Pistacho).
3. **Naranja** (Melón, mandarina).
4. Blanco (Pera)

Criterios de cobertura posibles:

1. Probar un caramelo de cada sabor.



Ejemplo: Cobertura de Caramelos

Tenemos **seis sabores**:

1. Limón.
2. Pistacho.
3. Melón.
4. Pera
5. Mandarina.
6. Damasco.

Tenemos **cuatro colores**:

1. **Amarillo** (Limón, damasco).
2. **Verde** (Pistacho).
3. **Naranja** (Melón, mandarina).
4. Blanco (Pera)

Criterios de cobertura posibles:

1. Probar un caramelo de cada sabor.
 - Decidir si el caramelo amarillo es de sabor limón o Damasco es un problema de controlabilidad.



Ejemplo: Cobertura de Caramelos

Tenemos **seis sabores**:

1. Limón.
2. Pistacho.
3. Melón.
4. Pera
5. Mandarina.
6. Damasco.

Tenemos **cuatro colores**:

1. **Amarillo** (Limón, damasco).
2. **Verde** (Pistacho).
3. **Naranja** (Melón, mandarina).
4. Blanco (Pera)

Criterios de cobertura posibles:

1. Probar un caramelo de cada sabor.
 - Decidir si el caramelo amarillo es de sabor limón o Damasco es un problema de controlabilidad.
2. Probar un caramelo de cada color.



Cobertura

Dado TR un conjunto de requisitos de test para un criterio de cobertura C , un conjunto de tests T satisface $C \iff \forall tr \in TR, \exists t \in T \mid t \text{ satisface } tr$

Ejemplo: Cobertura de caramelos



1. Amarillo (Limón, damasco).
2. Verde (Pistacho).
3. Naranja (Melón, mandarina).
4. Blanco (Pera)

Ejemplo: Cobertura de caramelos

$T_1 = \{ \text{tres de limón, una de pistacho, dos de melón, una de pera, una de mandarina, cuatro de damasco} \}$



1. Amarillo (Limón, damasco).
2. Verde (Pistacho).
3. Naranja (Melón, mandarina).
4. Blanco (Pera)

Ejemplo: Cobertura de caramelos

$T_1 = \{ \text{tres de limón, una de pistacho, dos de melón, una de pera, una de mandarina, cuatro de damasco} \}$

¿Satisface T_1 el criterio de sabor?

1. Amarillo (Limón, damasco).
2. Verde (Pistacho).
3. Naranja (Melón, mandarina).
4. Blanco (Pera)



Ejemplo: Cobertura de caramelos

$T_1 = \{ \text{tres de limón, una de pistacho, dos de melón, una de pera, una de mandarina, cuatro de damasco} \}$

¿Satisface T_1 el criterio de sabor?

$T_2 = \{ \text{una de limón, dos de pistacho, una de pera, tres de mandarina} \}$

1. Amarillo (Limón, damasco).
2. Verde (Pistacho).
3. Naranja (Melón, mandarina).
4. Blanco (Pera)



Ejemplo: Cobertura de caramelos

$T_1 = \{ \text{tres de limón, una de pistacho, dos de melón, una de pera, una de mandarina, cuatro de damasco} \}$

¿Satisface T_1 el criterio de sabor?

$T_2 = \{ \text{una de limón, dos de pistacho, una de pera, tres de mandarina} \}$

¿Satisface T_2 el criterio de sabor?

1. Amarillo (Limón, damasco).
2. Verde (Pistacho).
3. Naranja (Melón, mandarina).
4. Blanco (Pera)



Ejemplo: Cobertura de caramelos

$T_1 = \{ \text{tres de limón, una de pistacho, dos de melón, una de pera, una de mandarina, cuatro de damasco} \}$

¿Satisface T_1 el criterio de sabor?

$T_2 = \{ \text{una de limón, dos de pistacho, una de pera, tres de mandarina} \}$

¿Satisface T_2 el criterio de sabor?

¿Satisface T_2 el criterio de color?

1. Amarillo (Limón, damasco).
2. Verde (Pistacho).
3. Naranja (Melón, mandarina).
4. Blanco (Pera)



Cobertura

Requisitos de test inviables (infeasible): Aquellos que no se pueden satisfacer.

Cobertura

Requisitos de test inviables (infeasible): Aquellos que no se pueden satisfacer.

- ★ No existen valores para los tests que cumpla los requisitos.

- ★ Ejemplo: Código muerto.

- ★ La **detección** de requisitos inviables es **indecidable** para la mayoría de los criterios de testing.

Cobertura

Requisitos de test inviables (infeasible): Aquellos que no se pueden satisfacer.

- ★ No existen valores para los tests que cumpla los requisitos.

- ★ Ejemplo: Código muerto.

- ★ La **detección** de requisitos inviables es **indecidable** para la mayoría de los criterios de testing.

Por lo tanto, muchas veces es imposible conseguir 100% de cobertura en la práctica.

Grado de cobertura

Porcentaje de los requisitos de tests satisfechos por los tests de un conjunto.

Por ejemplo, T2 cumple cuatro de los seis requisitos del criterio “sabor”.

Como usar los criterios

Generar directamente los valores de test que satisfacen un criterio.

Asumido habitualmente por la comunidad científica.

Modo más obvio de usar los criterios.

Muy difícil de hacer sin usar herramientas.

Generar valores de test de forma externa y medir su bondad con respecto al criterio.

Usualmente utilizados en la industria.

¿Qué significa no alcanzar 100% de cobertura?

Como usar los criterios

Generar directamente los valores de test que satisfacen un criterio.

Asumido habitualmente por la comunidad científica.

Modo más obvio de usar los criterios.

Muy difícil de hacer sin usar herramientas.

Generar valores de test de forma externa y medir su bondad con respecto al criterio.

Usualmente utilizados en la industria.

¿Qué significa no alcanzar 100% de cobertura?

Los criterios de testing también se llaman *métricas*.

Generadores y reconocedores

Generador. Un procedimiento que genera automáticamente valores que satisfacen un criterio.

Generadores y reconocedores

Generador. Un procedimiento que genera automáticamente valores que satisfacen un criterio.

Reconocedor. Un procedimiento que decide si un conjunto de valores de tests satisfacen un criterio.

Generadores y reconocedores

Generador. Un procedimiento que genera automáticamente valores que satisfacen un criterio.

Reconocedor. Un procedimiento que decide si un conjunto de valores de tests satisfacen un criterio.

Usualmente es más fácil reconocer si un conjunto de valores satisface un criterio que generar valores que satisfagan un criterio.

Generadores y reconocedores

Generador. Un procedimiento que genera automáticamente valores que satisfacen un criterio.

Reconocedor. Un procedimiento que decide si un conjunto de valores de tests satisfacen un criterio.

Usualmente es más fácil reconocer si un conjunto de valores satisface un criterio que generar valores que satisfagan un criterio.

Afortunadamente, existen muchas herramientas para analizar cobertura.

Comparación de criterios: Subsumption

Subsumption. El criterio C_1 subsume a C_2 sii todos los conjunto de tests que satisface C_1 también satisface C_2 .

Importante: Se debe cumplir para **todos** los conjuntos de tests.

Comparación de criterios: Subsumption

Subsumption. El criterio C_1 subsume a C_2 sii todos los conjunto de tests que satisface C_1 también satisface C_2 .

Importante: Se debe cumplir para **todos** los conjuntos de tests.

Ejemplos:

Comparación de criterios: Subsumption

Subsumption. El criterio C_1 subsume a C_2 si todos los conjunto de tests que satisface C_1 también satisface C_2 .

Importante: Se debe cumplir para **todos** los conjuntos de tests.

Ejemplos:

- ★ El criterio de sabor subsume al criterio de color: si comemos de todos los sabores entonces comemos una de cada color.

Comparación de criterios: Subsumption

Subsumption. El criterio C_1 subsume a C_2 si todos los conjunto de tests que satisface C_1 también satisface C_2 .

Importante: Se debe cumplir para **todos** los conjuntos de tests.

Ejemplos:

- ★ El criterio de sabor subsume al criterio de color: si comemos de todos los sabores entonces comemos una de cada color.
- ★ Si un conjunto de tests cubre todas las ramas de un programa entonces cubre todas las sentencias.

Comparación de criterios: Subsumption

Generalmente, un criterio C_1 subsume a C_2 de dos maneras:

- ★ Los requisitos de test de C_1 siempre forman un superconjunto de los requisitos para C_2 .

Comparación de criterios: Subsumption

Generalmente, un criterio C_1 subsume a C_2 de dos maneras:

- ★ Los requisitos de test de C_1 siempre forman un superconjunto de los requisitos para C_2 .

Comparación de criterios: Subsumption

Generalmente, un criterio C_1 subsume a C_2 de dos maneras:

- ★ Los requisitos de test de C_1 siempre forman un superconjunto de los requisitos para C_2 .
- ★ Existe una correlación de muchos a uno (many-to-one) entre los requisitos de tests del criterio C_1 y los requisitos de tests del criterio C_2 . Si existe una correlación uno a uno (one to one) entre los requisitos de dos criterios, entonces se subsumen entre sí.

Comparación de criterios: Subsumption

Generalmente, un criterio C_1 subsume a C_2 de dos maneras:

- ★ Los requisitos de test de C_1 siempre forman un superconjunto de los requisitos para C_2 .
- ★ Existe una correlación de muchos a uno (many-to-one) entre los requisitos de tests del criterio C_1 y los requisitos de tests del criterio C_2 . Si existe una correlación uno a uno (one to one) entre los requisitos de dos criterios, entonces se subsumen entre sí.

Comparación de criterios: Subsumption

Generalmente, un criterio C_1 subsume a C_2 de dos maneras:

- ★ Los requisitos de test de C_1 siempre forman un superconjunto de los requisitos para C_2 .
- ★ Existe una correlación de muchos a uno (many-to-one) entre los requisitos de tests del criterio C_1 y los requisitos de tests del criterio C_2 . Si existe una correlación uno a uno (one to one) entre los requisitos de dos criterios, entonces se subsumen entre sí.

Algunas definiciones

Conjunto de Test minimal: Dado un conjunto de requisitos de test TR y un conjunto de test T que satisface todos los requisitos de test de TR , T es minimal si la eliminación de un test de T provocará que T ya no satisfaga todos los requisitos de test de TR .

Conjunto de Test mínimo: Dado un conjunto de requisitos de tests TR y un conjunto de tests T que satisface todos los requisitos de test de TR , T es mínimo si no hay un conjunto más pequeño de pruebas que también satisfaga todos los requisitos de test de TR .

Algunas definiciones

Conjunto de Test minimal: Dado un conjunto de requisitos de test TR y un conjunto de test T que satisface todos los requisitos de test de TR , T es minimal si la eliminación de un test de T provocará que T ya no satisfaga todos los requisitos de test de TR .

Conjunto de Test mínimo: Dado un conjunto de requisitos de tests TR y un conjunto de tests T que satisface todos los requisitos de test de TR , T es mínimo si no hay un conjunto más pequeño de pruebas que también satisfaga todos los requisitos de test de TR .

Algunas definiciones

Conjunto de Test minimal: Dado un conjunto de requisitos de test TR y un conjunto de test T que satisface todos los requisitos de test de TR , T es minimal si la eliminación de un test de T provocará que T ya no satisfaga todos los requisitos de test de TR .

Conjunto de Test mínimo: Dado un conjunto de requisitos de tests TR y un conjunto de tests T que satisface todos los requisitos de test de TR , T es mínimo si no hay un conjunto más pequeño de pruebas que también satisfaga todos los requisitos de test de TR .

Algunas definiciones

Conjunto de Test minimal: Dado un conjunto de requisitos de test TR y un conjunto de test T que satisface todos los requisitos de test de TR , T es minimal si la eliminación de un test de T provocará que T ya no satisfaga todos los requisitos de test de TR .

Conjunto de Test mínimo: Dado un conjunto de requisitos de tests TR y un conjunto de tests T que satisface todos los requisitos de test de TR , T es mínimo si no hay un conjunto más pequeño de pruebas que también satisfaga todos los requisitos de test de TR .

Algunas definiciones

Conjunto de Test minimal: Dado un conjunto de requisitos de test TR y un conjunto de test T que satisface todos los requisitos de test de TR , T es minimal si la eliminación de un test de T provocará que T ya no satisfaga todos los requisitos de test de TR .

Conjunto de Test mínimo: Dado un conjunto de requisitos de tests TR y un conjunto de tests T que satisface todos los requisitos de test de TR , T es mínimo si no hay un conjunto más pequeño de pruebas que también satisfaga todos los requisitos de test de TR .

Ventajas de diseñar tests basándonos en criterios

Ventajas de diseñar tests basándonos en criterios

Los criterios maximizan el beneficio del tiempo dedicado a testing.

Menos tests que son más efectivos para encontrar defectos.

Ventajas de diseñar tests basándonos en criterios

Los criterios maximizan el beneficio del tiempo dedicado a testing.

Menos tests que son más efectivos para encontrar defectos.

Amplios conjuntos de tests con mínimo solapamiento.

Ventajas de diseñar tests basándonos en criterios

Los criterios maximizan el beneficio del tiempo dedicado a testing.

Menos tests que son más efectivos para encontrar defectos.

Amplios conjuntos de tests con mínimo solapamiento.

Trazabilidad en el paso de artefactos software a tests.

Conocemos el “por qué” de cada test.

Nos proporciona un mecanismo para facilitar regression testing.

Ventajas de diseñar tests basándonos en criterios

Los criterios maximizan el beneficio del tiempo dedicado a testing.

Menos tests que son más efectivos para encontrar defectos.

Amplios conjuntos de tests con mínimo solapamiento.

Trazabilidad en el paso de artefactos software a tests.

Conocemos el “por qué” de cada test.

Nos proporciona un mecanismo para facilitar regression testing.

Tenemos una regla de parada: nos da una pista sobre el número de tests que necesitamos.

Características de un buen criterio de cobertura

Características de un buen criterio de cobertura

Debería ser relativamente sencillo calcular los requisitos de test automáticamente.

Características de un buen criterio de cobertura

Debería ser relativamente sencillo calcular los requisitos de test automáticamente.

Generar valores de tests debería poderse hacer eficientemente.

Características de un buen criterio de cobertura

Debería ser relativamente sencillo calcular los requisitos de test automáticamente.

Generar valores de tests debería poderse hacer eficientemente.

El conjunto de tests resultante debería mostrar tantos defectos como sea posible.

Cuatro estructuras para modelar software

Cuatro estructuras para
modelar software

```
graph TD; A[Cuatro estructuras para modelar software] --> B[Espacio de inputs]; A --> C[Grafos]; A --> D[Lógica]; A --> E[Syntaxis];
```

Espacio de
inputs

Grafos

Lógica

Syntaxis

